Atlas - Master Project Reference

Synchronized from docs/atlas_master.md | 2026-10-21

Version: 0.2.0 | Date: 2026-10-15 | Owner repository: Swayamjain1240/Atlas
Control status: ACCEPTED PRODUCT/STACK DECISIONS, NINE-PART ROADMAP, TARGET ARCHITECTURE; IMPLEMENTATION IN PROGRESS

1. Mission and product identity

Atlas is a local-first AI-powered Personal Life Operating System: a privacy-preserving index of user-approved personal documents, code and browser-history metadata, queried through local embeddings, RAG and Ollama with verifiable citations.
Success means a user can choose data sources, index them safely, ask source-grounded questions, inspect memories, manage/delete data, and use the UI offline after initial setup.

2. Source-of-truth hierarchy

- 1. This Master Reference: decisions, scope, status, milestones, Duck protocol and change control.
- 2. Requirements: acceptance IDs and release gates (functional, UX, security, non-functional).
- 3. System Design: endpoint contracts, schema, lifecycle, error format and test mapping.
- 4. Architecture: processes, responsibility boundaries, technology and deployment.
- 5. Actual repository code and executed test evidence: ground truth for what is implemented. Documentation NEVER proves implementation.
- For disagreement between these documents and implementation, raise a Duck finding before changing behavior. Preserve git history.

3. Fixed target stack

- Frontend: React 18, Vite, Tailwind CSS, Three.js and GSAP.
- Public backend: Node.js 20+ / Express with auth, validation, rate limiting, API contracts and SQLite ownership.
- Private AI: Python 3.11+ / internal FastAPI, Ollama + llama3.2:3b, Sentence-Transformers all-MiniLM-L6-v2, RAG, ChromaDB ownership.
- Persistence: SQLite metadata/sessions/jobs; ChromaDB vectors; user-approved local files.
- Security: local API keys, internal service authentication, strict localhost/CORS, validation, quotas, safe file access and dependency/secret scans.
- Architecture label: local-first modular multi-process monolith (single product, coordinated local processes).

4. Agreed top-level folders

- backend/ - Express code only after staged migration.
- frontend/ - React/Vite UI, Three.js graph and GSAP animations; reuse existing work.
- aiServices/ - Python AI/ingestion/retrieval stack; private loopback-only worker.
- tests/ - backend, frontend, aiServices, integration, E2E, security and performance.
- docs/ - these four control documents in PDF and editable Markdown formats.
- scripts/ - setup/start/test/audit scripts.
- data/ - local runtime-only SQLite, Chroma, caches; MUST be .gitignored. Prefer OS user-data location.

5. Verified repository baseline (observed 2026-10-15)

- Local branch main is at 11a6d8102cb0e2fb960ef3a871946101a73c30ac; verified origin/main is 341f5820f559d382b46b4ccef55970fe1552d343, one documentation-only commit ahead.
- A complete Git bundle of the pre-A.1 repository history was captured outside the working tree; no source, model, or personal data was deleted.

- Existing frontend React/Vite/Tailwind views are Chat, Dashboard, Timeline, Settings, Header, Sidebar and a Three.js particle background.
- The current frontend package has no GSAP dependency, no frontend test script and no tracked lockfile. It loads Google Fonts, which conflicts with the offline runtime requirement.
- Existing Python FastAPI is the public backend and exposes GET /, GET /api/v1/health, GET /api/v1/status and /openapi.json, plus auth/rate-limit/security helpers.
- backend/src/ai/engine.py is a stub: initialize only flips a flag, retrieve returns an empty list, generate returns a warming-up message, and stats reports zero documents.
- backend/src/ingestion and backend/src/storage contain only package initializer files; no functional ingestors, SQLite implementation or ChromaDB adapter exists.
- Chat calls POST /api/v1/query and Dashboard calls POST /api/v1/ingest, but those routes do not exist in the reviewed implementation.
- Timeline events are mock data; Dashboard values do not yet represent genuine persisted AI data; the graph renders random particles rather than indexed memories.
- The available Python smoke suite passes 18 tests; Python compilation passes. Frontend build and local Ollama inference were not executable in the audit environment because dependencies and Ollama were unavailable.
- Node/Express and aiServices are not implemented in the baseline; these documents specify target changes only.

6. Non-negotiable behavior

- Never silently upload local user content, run cloud inference, enable telemetry, or ingest a new data source without consent.
- Never delete the existing Python backend, installed models or personal data as a shortcut during migration.
- Auth and validation before ingestion, query, settings, memory access and deletion; every protected API call is traceable.
- Answer only using retrieved evidence where feasible; identify insufficient evidence; citations must resolve to real indexed chunks.
- Maintain one-viewport authenticated shell, responsive accessibility and optional reduced motion.
- Avoid changing chosen stack, high-priority requirements or scope without explicit user approval.

7. Nine-part development roadmap (dependencies and exit gates)

Migration A.1 - Existing project audit and legacy cleanup preparation

- Inspect the complete repository, branch state, dependencies, security controls, tests and runtime prerequisites.
- Preserve the current implementation and establish a recoverable rollback baseline.
- Inventory reusable Python, frontend and security modules; identify placeholders, obsolete responsibilities and architecture violations.
- Synchronize the editable and PDF control documents with the approved nine-part roadmap.
- Exit: the existing implementation is understood, recoverable and documented; no functional component or user data was deleted.

Migration A.2 - Legacy migration and folder restructuring

- Introduce the approved backend/, frontend/, aiServices/, tests/, docs/, scripts/ and runtime-data boundaries in staged form.
- Prepare the Express backend location and move reusable Python AI modules only after caller and test checks.
- Keep the legacy public Python backend available until replacement behavior is verified.
- Exit: the target directory structure is ready without regressions or data loss.

Rebuild B.1 - Node.js and Express backend

- Implement the Express gateway, environment validation, authentication, authorization, validation, rate limiting, safe errors, security headers and SQLite migration foundation.

- Add local-only health and status APIs and preserve stable versioned response contracts.
- Exit: the secure public backend foundation passes its unit, API, CORS and local-binding checks.

Rebuild B.2 - Python AI services

- Implement the private loopback FastAPI worker, internal authentication, Ollama adapter, model loading, Sentence-Transformers embeddings and Express-to-Python communication.
- Add timeouts, cancellation and safe model/service failure handling.
- Exit: local inference and 384-dimensional embedding generation work through authenticated internal contracts.

Rebuild B.3 - Databases and persistent memory

- Implement SQLite schemas, versioned migrations, ChromaDB collections, stable memory/chunk identifiers, conversation persistence and metadata access.
- Add cross-store reconciliation, restart recovery, deletion and backup/restore foundations.
- Exit: stored information survives restart, can be retrieved and can be deleted safely.

Rebuild B.4 - Document ingestion

- Implement explicit source consent, bounded file/code/PDF/DOCX/browser-history ingestion, safe path handling, chunking, embedding, indexing, deduplication and re-indexing.
- Add durable ingestion jobs, progress, retries, quotas and failure reporting.
- Exit: approved files become reliably searchable memories with traceable provenance.

Rebuild B.5 - RAG and AI chat

- Implement authenticated natural-language queries, semantic retrieval, context assembly, grounded Ollama generation, citations, relevance scores, multi-turn persistence, source filters and retrieval-only search.
- Defend against prompt injection, empty retrieval, model failure, timeout and unbounded-context scenarios.
- Exit: the complete React -> Express -> Python -> ChromaDB -> Ollama workflow works with verifiable citations.

Rebuild B.6 - Frontend, Three.js and GSAP

- Replace placeholder data with real API-backed dashboard, chat, timeline, memory management, settings and knowledge-graph views.
- Add lifecycle-safe GSAP animation, reduced-motion behavior, WebGL fallback, responsive accessibility, loading states and recoverable errors.
- Exit: complete frontend behavior works with real application data within the one-viewport contract.

Rebuild B.7 - Testing, hardening and release

- Complete unit, integration, API, contract, E2E, regression, security, accessibility, offline, backup/recovery and performance verification.
- Run dependency and secret scans, remove unused dependencies, validate production configuration and synchronize documentation.
- Exit: no unresolved critical failures remain and the complete local application passes release acceptance scenarios.

8. Duck checkpoint protocol

- Trigger word: Duck.
- Review the current git branch/head, staged/uncommitted changes, completed phase exit criteria, security controls and docs consistency.
- Output must include: completed work, remaining work, blockers, regression/security risks, decisions required, next safe step and GO / NO-GO.
- Never infer "complete" from a commit message, placeholder route, mock data or documentation.
- Before moving to a new phase, require evidence: tests + manual user-flow demonstration + no unresolved critical issues.

9. Work rules and engineering quality

- Small, meaningful commits. Prefer feature branches and PR review; direct main update only on explicit user request.
- Update requirements/architecture/design if changing interfaces, data schema, consent policy or scope.
- For any file changes, specify CREATE or UPDATE with exact path before implementation; do not generate duplicate masters.
- Test first for bug reproductions; retain regression test with every fix; no ignored test failures.
- Keep secrets/data out of Git. Use env examples with no real credentials. Do not expose private user paths in README/screenshots.
- Preserve original v0.1 PDFs in git history; replace current docs with v0.2, with versions and change rationale.

10. Open risks and explicit decisions

- Privacy: browser-accessible local endpoints need real credentials and origin/Host controls; localhost alone is insufficient.
- Cross-store consistency: SQLite and ChromaDB cannot share an atomic transaction; reconcile partial writes/deletes.
- Performance: p95 <2s full Ollama answer may not be realistic on CPU-only hardware; measure, do not promise.
- Deployment: model downloads may require internet at first install; package fonts and assets to enable truly offline runtime.
- Storage: define backup encryption, OS-specific user-data path and resource quotas before release.
- Product: email, cloud sync, MCP, autonomous agents, Tauri, team mode and calendar are OUT OF SCOPE until approved.

11. Part A.1 audit and rollback record

- Audit started only after explicit owner authorization on 2026-10-15; A.1 was completed before the owner explicitly authorized the staged A.2 migration on 2026-10-21.
- The pre-A.1 local HEAD, remote HEAD, branch relationship, repository status, source inventory, dependency inventory and available test results were recorded before documentation changes.
- The existing v0.1 PDFs remain recoverable through Git history and the captured repository bundle; the current working tree contains the verified v0.2 editable documents and architecture/system-design/requirements PDFs from origin/main, plus a regenerated master PDF synchronized from this file.
- Current verified checks: 18 legacy Python smoke tests passed; Python source compilation passed; pre-commit script and source/Markdown whitespace checks passed. The generic Git whitespace checker reports expected PDF cross-reference padding, so binary PDFs are excluded from that text check.
- Current verification gaps: frontend build is blocked by missing installed npm dependencies; Ollama, pip-audit and Bandit are unavailable; no E2E, integration, AI inference or performance verification exists yet.
- A.1 did not authorize deletion, replacement, commit, push, Part A.2, or rebuild implementation without a separate decision and verification. The owner later authorized A.2 with the explicit instruction duck and start a2; no commit or push has been performed.

A.1 migration inventory

- frontend/src/App.jsx and frontend/src/components/*: reusable presentation scaffold; preserve incrementally, replace API calls and placeholder data after Express contracts exist.
- frontend/src/styles/globals.css, frontend/tailwind.config.js and frontend/vite.config.js: reusable visual/dev configuration; review for offline assets, one-viewport behavior and Express proxy migration.
- frontend/package.json: retain React/Vite/Tailwind/Three.js dependencies; add GSAP, test tooling and a lockfile only in an authorized implementation change.
- backend/src/config.py: reusable environment/path concepts; migrate validation and secret handling into the Express configuration boundary without copying temporary-key behavior.
- backend/src/security.py: reusable security requirements and test ideas; review and reimplement authentication, CORS, headers, validation and limits in Express rather than treating the legacy FastAPI middleware as the target.

- backend/src/main.py: legacy public FastAPI application; keep available during migration, then retire public routes only after Express contracts and regression tests pass.
- backend/src/api/init.py: legacy response helper; use as a response-format reference and replace with the approved Express response contract.
- backend/src/ai/engine.py: placeholder only; do not treat its empty retrieval or warming-up response as reusable AI functionality.
- backend/src/ingestion/* and backend/src/storage/*: package initializers only; no implementation to preserve.
- backend/tests/*: 18-test legacy FastAPI smoke baseline; preserve as evidence, then translate or replace with centralized backend/security tests after the gateway exists.
- .env.example, .gitignore, README.md and scripts/pre-commit-hook.sh: project-control material requiring staged review for target paths, runtime data, reproducible dependencies, startup commands and security automation.
- docs/*: v0.2 architecture, system design, requirements and this master reference are target controls; PDFs and editable Markdown must remain synchronized.

12. Part A.2 migration record

- A.2 was authorized on 2026-10-21 after the Duck checkpoint reviewed the current Git state, rollback bundle, documentation, security gaps and executable evidence.
- The legacy public FastAPI service and its dependency file moved to backend/legacy_fastapi/; its 18-test smoke suite moved to tests/backend/legacy_fastapi/.
- The former backend/tests/ path remains as a compatibility wrapper so the pre-A.2 test command still exercises the canonical suite.
- backend/src/ now contains the Node.js/Express gateway configuration, security, route, middleware, utility and database foundations; backend/package.json records the Node 20+ runtime.
- aiServices/app/ now reserves the private Python worker module groups. The legacy AI engine is a documented placeholder and was not copied or promoted as working functionality.
- Top-level tests/ boundaries and the Git-ignored data/ runtime boundary were introduced. No source, model, personal data or legacy behavior was deleted.
- Verified A.2 checks: python -m pytest tests/backend/legacy_fastapi/ -v passed 18 tests; the compatibility command python -m pytest backend/tests/ -v passed 18 tests; Python compilation, the pre-commit script and non-PDF whitespace checks passed.
- A.2 exit status: complete for the staged directory and legacy-preservation boundary. The B.1 public gateway is implemented separately below; private AI worker behavior, ingestion, RAG, frontend integration and release gates remain future work in B.2-B.7.

13. Part B.1 implementation record

- B.1 was authorized after the owner requested completion with five to seven meaningful commits and a push to GitHub main.
- backend/src/config/env.js validates required credentials, loopback binding, exact CORS origins, ports, limits and local data paths without generating fallback secrets.
- backend/src/middleware/* provides request IDs, safe errors, Helmet headers, host/origin checks, constant-time bearer authentication, scope authorization, validation and bounded rate limiting.
- backend/src/database/* provides the Node-owned SQLite migration runner, WAL/foreign-key setup, integrity status and the initial sources, memories, jobs, chat, settings and schema-migrations tables.
- backend/src/routes/*, backend/src/app.js and backend/server.js provide the public health endpoint and authenticated status endpoint on loopback with the standard versioned response contract.
- Verified B.1 tests: npm test passed 9 Node tests covering configuration failure modes, migration idempotence, health/status contracts, auth, secret non-disclosure, host/origin checks, rate limiting, safe JSON errors and unknown routes.
- B.1 exit status: complete for the secure gateway foundation. Query, ingestion, private Python communication, ChromaDB, RAG, frontend data integration and release gates remain B.2-B.7 work.

14. Immediate next action

- Preserve the legacy FastAPI service until the replacement workflow is verified.
- Run the next Duck checkpoint before beginning B.2; do not silently advance into AI-worker implementation.